EMBEDDED ONBOARDING · CONTROL TEAM

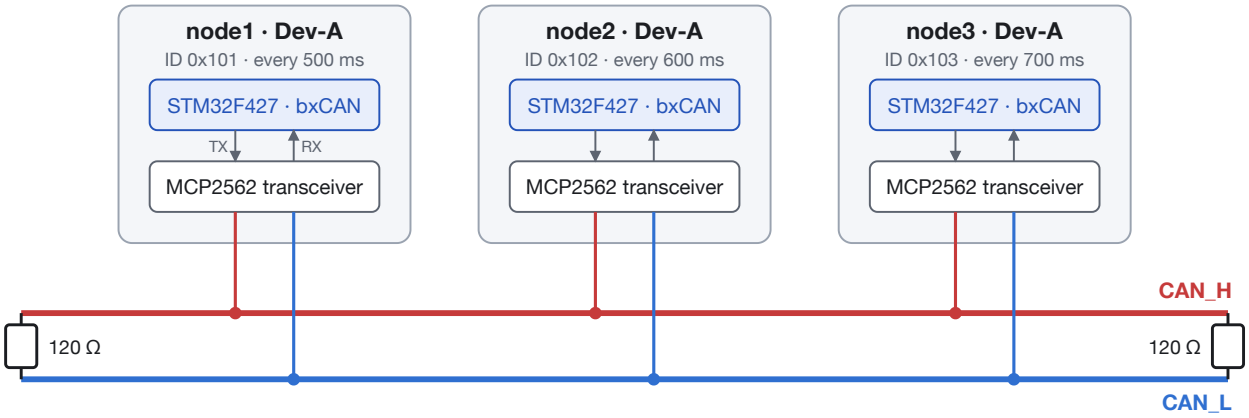
CAN Bus Basics

Three simulated RoboMaster Dev-A boards talking over one CAN bus. The setup is done for you; you write the four functions that make the boards talk, using the STM32 HAL, and test them in the Renode simulator. No hardware needed until chapter 9.

TIME
About 2 hours

YOU NEED
A GitHub account (for Codespaces) or a computer with Renode and the ARM compiler

YOU'LL LEARN
GPIO, CAN frames, IDs, filters and interrupts, and how to send and receive with the STM32 HAL



What you'll build. Three boards run the same firmware and share one CAN bus. In the simulator, the wires, transceivers and resistors are replaced by a single virtual “CAN hub”.

1 What you'll build
2 CAN in ten minutes
3 Set up
4 First run: the starter project
5 Tour of the project
6 Your tasks
7 Challenges
8 Check your work automatically
9 From the simulator to real boards
10 Troubleshooting
A DJI motor CAN frames
B Renode cheat sheet

1 What you'll build

On the robot, the main controller talks to the motors, the gimbal and the other boards over **CAN**. Before you touch a real robot, you'll learn CAN on three *simulated* RoboMaster Development Board Type A ("Dev-A", STM32F427IIH6) boards.

Each board runs the **same firmware**. The only difference is a number, `NODE_ID` (1, 2 or 3). When you've finished the four tasks in chapter 6, every board:

1. sends a small **heartbeat** frame on the bus at its own period, and
2. prints every frame it receives on its serial port (USART6) and shows a counter from the frame in binary on its 8 LEDs.

Board	CAN ID it sends	Period	Data bytes
node1	`0x101`	500 ms	`01` , counter
node2	`0x102`	600 ms	`02` , counter
node3	`0x103`	700 ms	`03` , counter

Why a simulator?

Renode is an open-source simulator from Antmicro. It runs the exact `.elf` file you would flash onto a board, with the CPU, timers, GPIO, UART and CAN controller emulated. You can run three boards, connect them to one virtual bus and watch every frame without wiring anything or bricking a board.

> **KEEP IN MIND**
>
> A simulator is not the real thing. Renode does not simulate wiring, pin choice, bit timing or bus errors. Chapter 9 lists exactly what changes on real boards. Read it before your first hardware session.

What's in the repository

```
can-bus-onboarding/
├── Makefile                  make · make sim · make test
├── firmware/
│   ├── Core/Src/main.c       ← the file you will edit
│   ├── Core/Src/stm32f4xx_hal_msp.c   pins, clocks and interrupts for CAN1 and USART6
│   ├── Core/Src/stm32f4xx_it.c        interrupt handlers
│   ├── Core/Inc/…            headers, HAL configuration
│   ├── Drivers/              ST's HAL and CMSIS (unchanged)
│   └── STM32F427IIHX_FLASH.ld         linker script
├── sim/
│   ├── dev-a.repl            the simulated Dev-A board
│   ├── can_bus.resc          three boards + one CAN bus
│   └── can_bus.robot         the automatic test
├── docs/                     this guide
└── .devcontainer/            setup for GitHub Codespaces
```

The layout matches a project generated by STM32CubeMX, so what you learn here carries straight over to your team's real code. Chapter 5 explains what every file does.

2 CAN in ten minutes

2.1 One pair of wires, many boards

A CAN bus is two wires, **CAN_H** and **CAN_L**, usually twisted together. Every board connects to the same two wires, in parallel. A **120 Ω resistor at each end** of the bus stops signals from reflecting back.

The bus only has two states:

- **Dominant = logic 0.** A transmitter pushes CAN_H up and CAN_L down.
- **Recessive = logic 1.** Nobody drives the bus; the resistors pull CAN_H and CAN_L to the same voltage.

If any one board sends a 0, the whole bus reads 0. **“0 wins”** is the most important rule in CAN; it makes arbitration and ACK work (see 2.5 and 2.6).

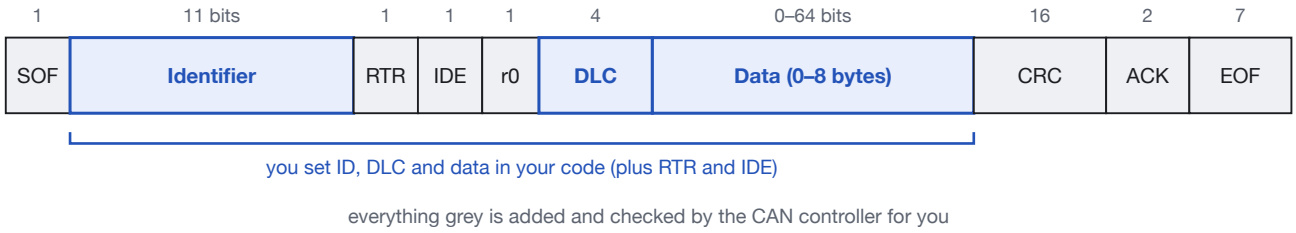
Because the receiver looks at the *difference* between the two wires, noise that hits both wires equally cancels out. That's why CAN survives next to motors.

2.2 Inside each board

- The **CAN controller** lives inside the microcontroller. On the STM32F4 it's called **bxCAN** (there are two: CAN1 and CAN2). It builds frames, handles timing, CRC, ACK and retries, and filters incoming frames. Your code only ever talks to the controller, through the HAL.
- The **transceiver** is a separate chip (an MCP2562 on the Dev-A). It converts the controller's TX/RX logic pins into CAN_H/CAN_L voltages and back.

bxCAN has **3 transmit mailboxes** (frames waiting to go out), **2 receive FIFOs** that hold 3 frames each, and **28 filter banks** shared between CAN1 and CAN2.

2.3 A CAN frame



A standard CAN data frame. At 1 Mbit/s an 8-byte frame takes about 0.13 ms.

Field	Meaning	In the HAL
Identifier	11-bit number (0x000–0x7FF) that says *what* the message is. Also its priority.	`header.StdId`
RTR	Data frame (normal) or remote request (rarely used).	`header.RTR = CAN_RTR_DATA`
IDE	Standard 11-bit ID or extended 29-bit ID.	`header.IDE = CAN_ID_STD`
DLC	Data Length Code: how many data bytes follow (0–8).	`header.DLC`
Data	Up to 8 bytes. What they mean is up to you (or the motor's manual).	`uint8_t data[8]`

2.4 Everyone hears everything

CAN has **no addresses**. When a board sends a frame, *every* other board on the bus receives it. The ID tells receivers what the message is about; each receiver decides whether it cares. The CAN controller can do that sorting in hardware with **filters**, so your code never sees frames it doesn't want. A board never receives its own frames.

2.5 IDs are priorities: arbitration

What if two boards start sending at the same moment? Both send their ID bit by bit, most significant bit first, and both listen to the bus while they send. Because 0 wins, as soon as one board sends a 1 but reads back a 0, it knows someone with a **lower ID** is also sending. It stops and tries again after the bus is free. The winner never notices and no data is lost.

Bit	10	9	8	7	6	5	4	3	2	1	0
node1 sends 0x101	0	0	1	0	0	0	0	0	0	**0**	1
node2 sends 0x102	0	0	1	0	0	0	0	0	0	1 ✗	
bus	0	0	1	0	0	0	0	0	0	**0**	1

The lower the ID, the higher the priority. That's why DJI motor *commands* use ID 0x200 and the motors' *feedback* uses 0x201 and up.

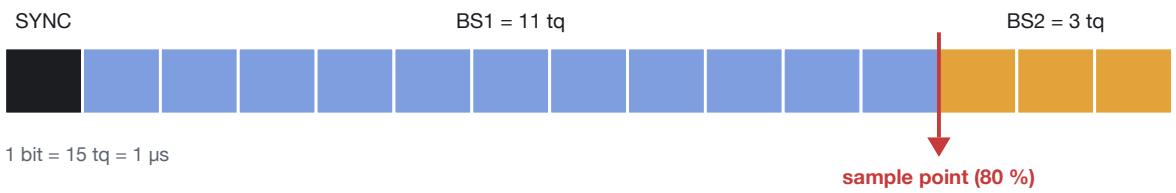
2.6 ACK: somebody has to answer

In the ACK slot of every frame the sender transmits a 1, and every receiver that got the frame correctly overwrites it with a 0. If the sender reads back a 1, nobody received the frame. It reports an error and sends again, forever.

> **COMMON FIRST-DAY BUG**
>
> A board alone on the bus never gets an ACK, so its 3 transmit mailboxes fill up and stay full. Same if the other board isn't running, or uses a different bit rate. In Task 3 you'll make your firmware print `tx skipped: all 3 mailboxes busy` when that happens.

2.7 Bit rate and bit timing

All boards on a bus must use the **same bit rate**. RoboMaster robots use **1 Mbit/s**, the rate DJI motors and ESCs expect. The controller splits each bit into small time slices called **time quanta (tq)** and reads the bus at the **sample point**, near the end of the bit:



The bit timing used in this project: 45 MHz ÷ 3 = 15 MHz, so 1 tq = 66.7 ns; 15 tq = 1 µs per bit = 1 Mbit/s.

You'll see how these numbers are set in code in section 5.3.

3 Set up

You need two tools: the **ARM GCC compiler** (`arm-none-eabi-gcc`) to build the firmware, and **Renode 1.17** to run it. Pick one option.

Option A: GitHub Codespaces (recommended, nothing to install)

A Codespace is a Linux computer in the cloud with VS Code in your browser. The repository tells it what to install.

1. Open the repository on GitHub: github.com/bernininini/can-bus-onboarding (or your team's copy). If you see a 404, ask your lead to give you access.
2. Click the green **Code** button → **Codespaces** tab → **Create codespace on main**.
3. Wait for the setup to finish. The first time takes 3–5 minutes; the terminal at the bottom shows `.devcontainer/setup.sh` installing things. It's done when it prints:

   ```
   arm-none-eabi-gcc (15:13.2.rel1-2) 13.2.1 20231009
   Renode v1.17.0
   ```

4. Click into the terminal and carry on with chapter 4.

> **GOOD TO KNOW**
>
> Personal GitHub accounts get a free monthly Codespaces allowance (enough for many hours on the default 2-core machine). When you're done, stop the codespace from github.com/codespaces; it also stops by itself after 30 minutes of inactivity. Your files are kept.

Option B: your own computer

Linux (Ubuntu 24.04) or Windows (WSL 2 with Ubuntu 24.04)

On Windows, first install WSL: open PowerShell as administrator, run `wsl --install -d Ubuntu-24.04` , restart, and open “Ubuntu” from the Start menu. Then, in the Ubuntu terminal:

```
git clone https://github.com/bernininini/can-bus-onboarding.git
cd can-bus-onboarding
bash .devcontainer/setup.sh       # installs gcc-arm-none-eabi and Renode (asks for your password)
```

macOS (Apple Silicon)

1. Download `renode-1.17.0.osx-arm64-portable.dmg` from github.com/renode/renode/releases/tag/v1.17.0, open it and drag **Renode.app** into Applications.
2. Install the ARM compiler: either `brew install --cask gcc-arm-embedded` , or download `arm-gnu-toolchain-14.3.rel1-darwin-arm64-arm-none-eabi.tar.xz` from developer.arm.com and unpack it.
   The compiler that ships with STM32CubeIDE/CubeCLT is built for Intel Macs and only runs on Apple Silicon if Rosetta is installed.
3. Put both tools on your `PATH` , the list of folders your shell searches for commands, by adding two lines to `~/.zshrc` ; then open a new terminal:

```
echo 'export PATH="/Applications/Renode.app/Contents/MacOS:$PATH"' >> ~/.zshrc
echo 'export PATH="$HOME/arm-gnu-toolchain-14.3.rel1-darwin-arm64-arm-none-eabi/bin:$PATH"'
>> ~/.zshrc
```

Check with `which -a arm-none-eabi-gcc` : the first line must be the one you just installed. (If you also have STM32CubeCLT, its Intel-only compiler appears further down; the first match wins.)

4. Clone the repository:

```
git clone https://github.com/bernininini/can-bus-onboarding.git
cd can-bus-onboarding
```

MACOS SECURITY PROMPT

If macOS says Renode “can't be opened because Apple cannot check it”, right-click Renode.app → **Open** once, or allow it under System Settings → Privacy & Security.

4 First run: the starter project

The project you cloned is a **starter**. Everything CubeMX would generate is done: clocks, pins, the CAN1 bit rate, the serial port, `printf`. Four small functions are left empty for you. Before you write anything, build and run it so you know what "nothing done yet" looks like.

1. **Build the firmware.** From the repository folder:

   ```
   make
   ```

   This compiles the same code three times, once with `-DNODE_ID=1`, once with 2 and once with 3, and ends with:

   ```
   LD  build/node3.elf
      text    data     bss     dec     hex filename
     11632     112    2312   14056    36e8 build/node3.elf
   ```

   You now have `build/node1.elf`, `node2.elf` and `node3.elf`: one file per board. Run `make` again and it says `Nothing to be done`: it only rebuilds files that changed.

2. **Start the simulation.**

   ```
   make sim
   ```

   Renode loads `sim/can_bus.resc`, which creates three Dev-A boards, loads one `.elf` into each, connects all three CAN1 controllers to one virtual bus, and starts them. The first run downloads a small register-description file, so you need internet.

3. **Look at the output.** After a few seconds:

   ```
   node1/usart6: [… virt: 0.1ms …] [node1] up: sending id=0x101 every 500 ms
   node2/usart6: [… virt: 0.1ms …] [node2] up: sending id=0x102 every 600 ms
   node3/usart6: [… virt: 0.1ms …] [node3] up: sending id=0x103 every 700 ms
   ```

   And then nothing. The boards boot and announce themselves, but nobody sends anything yet: that's your job.

4. **Stop it.** Type `quit` and press `Enter` (the prompt shows `(node3)`), or press `Ctrl`+`C`.

5. **Run the automatic check.**

   ```
   make test
   ```

   Every test fails, because no task is done yet. Each one is named after the task it checks, so as you work through chapter 6 you'll see them turn `OK` one by one. (On a Mac, chapter 8 shows a one-time setup step for `make test`.)

How to read a log line

```
[19:24:12.4628] [INFO] node2/usart6: [host: 1.47s (+1.15s)|virt: 0.5s (+0.5s)] [node2] rx id=0x101
dlc=2 data: 01 00
```

Part	Meaning
`[19:24:12.4628] [INFO]`	Clock on your computer, and the log level
`node2/usart6:`	Which board, and which UART the text came out of
`host: 1.47s`	Real time that has passed on your computer. The simulator may run slower or faster than real life; that's fine.
`virt: 0.5s`	**Simulated time**, which is what the firmware sees. Use it to check periods: node1 sends at 0.5 s, 1.0 s, 1.5 s… The `(+…)` is the gap since this UART's previous line.
`[node2] rx id=0x101 …`	What the firmware printed with `printf()`

HARMLESS WARNINGS

At startup you'll see a few lines like `WARNING … flash_controller: Unhandled write … ICEN` and `rcc: Unhandled write … RESERVED`. They mean Renode doesn't model the flash cache or one reserved bit. Ignore them.

Peek at the LEDs

Renode has no picture of the board, but you can ask for any LED's state. While the simulation runs, type into the Renode prompt:

```
pause
mach set "node3"
gpioPortG.led_a State
gpioPortG.led_b State
start
```

Each `State` line prints `True` (on) or `False` (off). Right now every LED is off. After Task 1, node3 will show `True`, `True` : 3 in binary.

5 Tour of the project

Read this chapter once before you start the tasks. It tells you where everything is, what's already done, and which HAL functions you'll need.

5.1 What's in each file

File	What's in it	You edit it?
`firmware/Core/Src/main.c`	`main()`, the setup functions CubeMX would generate, your four **TASK** functions, and some helpers	**Yes**, only the TASK functions
`firmware/Core/Inc/main.h`	LED pin names: `LED_A_Pin` … `LED_H_Pin` (PG1–PG8) and `LED_GPIO_Port`	No (read it)
`firmware/Core/Src/stm32f4xx_hal_msp.c`	Low-level setup: turns on the CAN1 and USART6 clocks, puts CAN1 on pins PD0/PD1, enables the CAN1 RX0 interrupt	No (read it)
`firmware/Core/Src/stm32f4xx_it.c`	Interrupt handlers. `CAN1_RX0_IRQHandler()` hands every CAN receive interrupt to the HAL	No
`firmware/Core/Src/syscalls.c`, `system_stm32f4xx.c`, `Core/Startup/`	C library glue, startup code that runs before `main()`	No
`firmware/Drivers/`	ST's HAL library and CMSIS headers: the code behind every `HAL_…` function	No, but worth reading
`sim/dev-a.repl`, `sim/can_bus.resc`	The simulated board, and the script that builds three of them on one bus	No
`sim/can_bus.robot`	The automatic check (`make test`)	No

5.2 A map of `main.c`

`main.c` is laid out like a file CubeMX generates. CubeMX owns everything outside the `/* USER CODE BEGIN … */` / `/* USER CODE END … */` markers; code between them survives when the project is regenerated. From top to bottom:

Part	What it does
`#define NODE_ID`, `HEARTBEAT_ID`, `HEARTBEAT_PERIOD_MS`	The node's number (set by the Makefile), the CAN ID it sends (0x100 + NODE_ID) and how often (400 + 100 × NODE_ID ms)
`hcan1`, `huart6`	The HAL "handles" for CAN1 and USART6. Every HAL CAN function takes `&hcan1`.
`main()`	Runs the setup, calls `show_on_leds(NODE_ID)` and `can_start()`, prints the `up:` line, then loops forever: calls `send_heartbeat()` on a timer and prints every frame that was received
`show_on_leds()` · `can_start()` · `send_heartbeat()` · `HAL_CAN_RxFifo0MsgPendingCallback()`	**Your four tasks.** Each has a comment block above it saying what to do and which functions to use.
`print_tx()` · `print_rx()` · `rx_push()` · `rx_pop()`	Helpers that are already written (5.5)
`SystemClock_Config()`, `MX_CAN1_Init()`, `MX_USART6_UART_Init()`, `MX_GPIO_Init()`	The CubeMX-generated setup (5.3)

Here is the main loop, so you can see where your functions get called:

```
while (1)
{
  /* Every HEARTBEAT_PERIOD_MS milliseconds, send one heartbeat. */
  if ((int32_t)(HAL_GetTick() - next_tx) >= 0)
  {
    next_tx += HEARTBEAT_PERIOD_MS;
    send_heartbeat(counter++);                  /* ← Task 3 */
  }

  /* Print every frame the receive callback has queued up. */
  RxFrame f;
  while (rx_pop(&f))                            /* ← filled by Task 4 */
  {
    print_rx(&f.header, f.data);
    if (f.header.DLC >= 2)
    {
      show_on_leds(f.data[1]);                  /* ← Task 1 */
    }
  }
}
```

`HAL_GetTick()` counts milliseconds since boot. Writing the timer check as a subtraction keeps it working when the 32-bit counter wraps around after 49 days.

5.3 What's already set up for you

The clock

`SystemClock_Config()` takes the Dev-A's **12 MHz crystal** through the PLL up to **180 MHz**:

Clock	Setting	Result
PLL	12 MHz ÷ M(6) × N(180) ÷ P(2)	180 MHz system clock
APB1	180 MHz ÷ 4	**45 MHz, which feeds CAN1 and CAN2**
APB2	180 MHz ÷ 2	90 MHz, which feeds USART6

CAN1 and its bit rate

```
static void MX_CAN1_Init(void)
{
  hcan1.Instance = CAN1;
  hcan1.Init.Prescaler = 3;
  hcan1.Init.Mode = CAN_MODE_NORMAL;
  hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
  hcan1.Init.TimeSeg1 = CAN_BS1_11TQ;
  hcan1.Init.TimeSeg2 = CAN_BS2_3TQ;
  hcan1.Init.AutoBusOff = ENABLE;
  hcan1.Init.AutoRetransmission = ENABLE;
  …
  HAL_CAN_Init(&hcan1);
}
```

```
bit rate = APB1 clock / Prescaler / (1 + BS1 + BS2)
         = 45 MHz     / 3         / (1 + 11 + 3)    = 1 000 000 bit/s
sample point = (1 + BS1) / (1 + BS1 + BS2) = 12 / 15 = 80 %
```

`HAL_CAN_Init()` leaves the controller in **initialization mode**: configured, but off the bus. Joining the bus is Task 2.

Pins and the interrupt

`HAL_CAN_Init()` calls `HAL_CAN_MspInit()` in `stm32f4xx_hal_msp.c`, which puts CAN1 on **PD0** (RX, with a pull-up) and **PD1** (TX). Those come from the Dev-A schematic, not CubeMX's default of PA11/PA12. It also enables the `CAN1_RX0` interrupt in the NVIC. When that interrupt fires, `CAN1_RX0_IRQHandler()` in `stm32f4xx_it.c` calls `HAL_CAN_IRQHandler(&hcan1)`, which works out why and calls your Task 4 callback.

printf

`printf()` doesn't know about UARTs. The C library calls `_write()` to output characters, and `main.c` provides one that sends them out of USART6. In the simulator, Renode prints whatever arrives on USART6 into its log.

5.4 The HAL functions you'll use

Every one returns `HAL_OK` on success (except the mailbox counter, which returns a number). If a setup call doesn't return `HAL_OK`, call `Error_Handler()`, which stops the board so you notice.

Function	What it does	Task
`HAL_GPIO_WritePin(port, pin, state)`	Drives a pin high (`GPIO_PIN_SET`) or low (`GPIO_PIN_RESET`)	1
`HAL_CAN_ConfigFilter(&hcan1, &filter)`	Sets up one filter bank from a `CAN_FilterTypeDef`	2
`HAL_CAN_Start(&hcan1)`	Leaves initialization mode and joins the bus	2
`HAL_CAN_GetTxMailboxesFreeLevel(&hcan1)`	How many of the 3 transmit mailboxes are free (0–3)	3
`HAL_CAN_AddTxMessage(&hcan1, &header, data, &mailbox)`	Puts a frame in a free mailbox. Returns right away; the controller sends it as soon as the bus is free. `mailbox` is a `uint32_t` it fills in.	3
`HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING)`	Turns on the interrupt for "a frame arrived in FIFO0"	4
`HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &header, data)`	Copies the oldest frame out of FIFO0 into your header and `uint8_t data[8]` , and frees its slot	4

READ THE SOURCE

The HAL documents itself. Open `firmware/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_can.c` : lines 28–250 ("How to use this driver") describe the whole CAN workflow, and the comment above each function explains its parameters. In VS Code, `Cmd` / `Ctrl` -click any `HAL_…` name to jump to it.

The structs you'll fill in

`CAN_FilterTypeDef` (Task 2):

Field	What to put
`FilterBank`	Which of the 28 banks to set up. Use `0`.
`FilterMode`	`CAN_FILTERMODE_IDMASK` (an ID plus a mask) or `CAN_FILTERMODE_IDLIST` (a list of exact IDs)
`FilterScale`	`CAN_FILTERSCALE_32BIT` (one 32-bit filter) or `CAN_FILTERSCALE_16BIT` (two 16-bit ones)
`FilterIdHigh`, `FilterIdLow`	The ID bits to compare against (high and low 16 bits)
`FilterMaskIdHigh`, `FilterMaskIdLow`	In mask mode: which bits must match. A **1** means "must equal the ID bit", a **0** means "don't care".
`FilterFIFOAssignment`	Where accepted frames go: `CAN_FILTER_FIF00`
`FilterActivation`	`ENABLE`
`SlaveStartFilterBank`	Where CAN2's banks start; the usual split is `14` (0–13 for CAN1, 14–27 for CAN2)

`CAN_TxHeaderTypeDef` (Task 3): `StdId` (the 11-bit ID), `IDE` (`CAN_ID_STD` for 11-bit IDs), `RTR` (`CAN_RTR_DATA` for a normal data frame), `DLC` (number of data bytes, 0–8), `TransmitGlobalTime` (`DISABLE`).

`CAN_RxHeaderTypeDef` (Task 4): the same fields, filled in *for* you by `HAL_CAN_GetRxMessage()`.

Tip: start every struct with `= {0}` (e.g. `CAN_TxHeaderTypeDef header = {0};`) so fields you don't set are zero instead of random.

5.5 Helpers that are already written

Function	What it does
`print_tx(&header, data)`	Prints `[node1] tx id=0x101 dlc=2 data: 01 00`. Call it in Task 3 after sending.
`print_rx(&header, data)`	Prints `[node2] rx id=0x101 …`. The main loop already calls it.
`rx_push(&header, data)`	Puts a received frame in a small queue. Call it in Task 4.
`rx_pop(&frame)`	Takes a frame out of the queue. The main loop already calls it.

6 Your tasks

Four tasks, each one function in `firmware/Core/Src/main.c`. Do them in order: each builds on the one before. For every task:

1. Read the comment block above the function in `main.c` and the section here.
2. Replace the `/* TODO */` (and the `(void)…;` line under it, which only exists to silence a compiler warning) with your code.
3. Run `make sim` and compare with “Check”. Then run `make test` and look for the task's name.

> **UNDO EVERYTHING**
> To get the original `main.c` back at any time: `git checkout firmware/Core/Src/main.c`. To keep your work but start fresh: `git stash`. Commit after each task that works (`git commit -am "Task 1"`), so you can always go back.

Task 1: LEDs (GPIO warm-up)

Goal: `show_on_leds(value)` shows `value` in binary on the 8 user LEDs, LED_A = bit 0 up to LED_H = bit 7. `main()` calls it at boot with `NODE_ID`, so node3 should light LED_A and LED_B (3 = `00000011`).

You'll use: `HAL_GPIO_WritePin(port, pin, state)`, `LED_GPIO_Port`, `LED_A_Pin` … `LED_H_Pin`.

What you need to know

- **The LEDs are active-low.** On the Dev-A each LED sits between 3.3 V (through a 1 kΩ resistor) and its pin. Current only flows when the pin is *low*, so `GPIO_PIN_RESET` turns an LED **on** and `GPIO_PIN_SET` turns it off.
- **Pins are bit masks.** `LED_A_Pin` is `GPIO_PIN_1`, which is just the number `1 << 1`; `LED_B_Pin` is `1 << 2`, and so on. Because the 8 LEDs are on neighbouring pins (PG1–PG8), `LED_A_Pin << n` is the pin for LED number *n*.
- **Reading one bit.** In C, `(x >> n) & 1` is bit *n* of `x`. Example: 6 is `110` in binary, so `(6 >> 0) & 1` is 0 and `(6 >> 1) & 1` is 1.

Steps

1. Loop over the 8 bits.
2. For each bit, decide on or off, and remember the LEDs are active-low.
3. Write that state to the matching pin with `HAL_GPIO_WritePin()`.

Check

The log doesn't change, so look at the LEDs (section 4, “Peek at the LEDs”). On node3, `gpioPortG.led_a State` and `led_b` should print `True` and `led_c` `False`. Then `make test`: **Task 1 - LEDs Show NODE_ID At Boot** should be `OK`.

> **COMMON MISTAKES**
> - Everything is backwards (node3 lights LED_C to LED_H): you forgot the LEDs are active-low.
> - Only LED_A ever changes: you wrote to `LED_A_Pin` every time instead of shifting it.

Task 2: Start CAN1

Goal: `can_start()` brings CAN1 onto the bus, ready to accept every frame into receive FIFO0.

You'll use: `CAN_FilterTypeDef`, `HAL_CAN_ConfigFilter()`, `HAL_CAN_Start()`, `Error_Handler()`.

What you need to know

- After `MX_CAN1_Init()`, CAN1 knows its bit rate but is still in **initialization mode**: off the bus.
- **Until at least one filter is active, bxCAN throws every received frame away.** A filter decides which IDs to keep and which FIFO they go into.
- In **mask mode** a filter has an ID and a mask. For every mask bit that is 1, the received ID bit must equal the filter's ID bit; mask bits that are 0 mean “don't care”. So a mask of all zeros means “don't care about anything”: every frame is accepted.

Steps

1. Make a `CAN_FilterTypeDef` (start it with `= {0}`) and fill in every field from the table in section 5.4: bank 0, mask mode, 32-bit scale, ID and mask all zero, FIFO0, enabled, `SlaveStartFilterBank = 14`.
2. Call `HAL_CAN_ConfigFilter(&hcan1, &filter)`. If it doesn't return `HAL_OK`, call `Error_Handler()`.
3. Call `HAL_CAN_Start(&hcan1)`, with the same check.

Check

Still nothing new in the log, because nothing is being sent yet. `make test`: **Task 2 - CAN1 Is Started With A Filter** should be `OK`. That test reads two CAN1 registers directly: `CAN_MCR` (has the controller left initialization mode?) and `CAN_FA1R` (is filter bank 0 active?).

COMMON MISTAKES

- The `up:` lines don't appear any more: the board stopped in `Error_Handler()` before printing, because one of your calls didn't return `HAL_OK`. Check the filter fields.
- `make test` says CAN1 is still in initialization mode: `HAL_CAN_Start()` isn't being called, or you return before reaching it.

Task 3: Send a frame

Goal: `send_heartbeat(counter)` sends one frame with ID `HEARTBEAT_ID` and 2 data bytes: `NODE_ID`, then `counter`. `main()` already calls it on a timer.

You'll use: `CAN_TxHeaderTypeDef`, `HAL_CAN_AddTxMessage()`, `HAL_CAN_GetTxMailboxesFreeLevel()`, `print_tx()`.

What you need to know

- The **header** describes the frame: `StdId`, `IDE`, `RTR`, `DLC` (section 2.3 shows where each sits in a frame). The **data** is a separate `uint8_t` array.
- `HAL_CAN_AddTxMessage()` doesn't wait for the frame to go out. It puts it in one of 3 transmit **mailboxes** and returns; the controller sends it as soon as the bus is free and retries on its own if needed. Its last argument is the address of a `uint32_t` where it writes which mailbox it used.

- On a real bus, if nobody acknowledges your frames (2.6), they stay in the mailboxes and all 3 fill up. Checking `HAL_CAN_GetTxMailboxesFreeLevel()` first lets you print a clear message instead of failing silently.

Steps

1. Make a `CAN_TxHeaderTypeDef` and set `StdId`, `IDE`, `RTR`, `DLC` and `TransmitGlobalTime`.
2. Make a 2-byte `uint8_t` array with `NODE_ID` and `counter`.
3. If no mailbox is free, print `tx skipped: all 3 mailboxes busy` and return.
4. Call `HAL_CAN_AddTxMessage()`. If it returns `HAL_OK`, call `print_tx(&header, data)`; otherwise print that it failed.

Check

`make sim` now shows every board sending, on its own period:

```
[node1] tx id=0x101 dlc=2 data: 01 00
[node2] tx id=0x102 dlc=2 data: 02 00
[node3] tx id=0x103 dlc=2 data: 03 00
[node1] tx id=0x101 dlc=2 data: 01 01
[node2] tx id=0x102 dlc=2 data: 02 01
```

There are no `rx` lines yet. The frames reach the other boards' FIFOs, but nothing reads them out: that's Task 4. `make test`: **Task 3 - Each Node Sends Its Heartbeat** should be `OK`. That test watches the bus itself, not the log.

COMMON MISTAKES

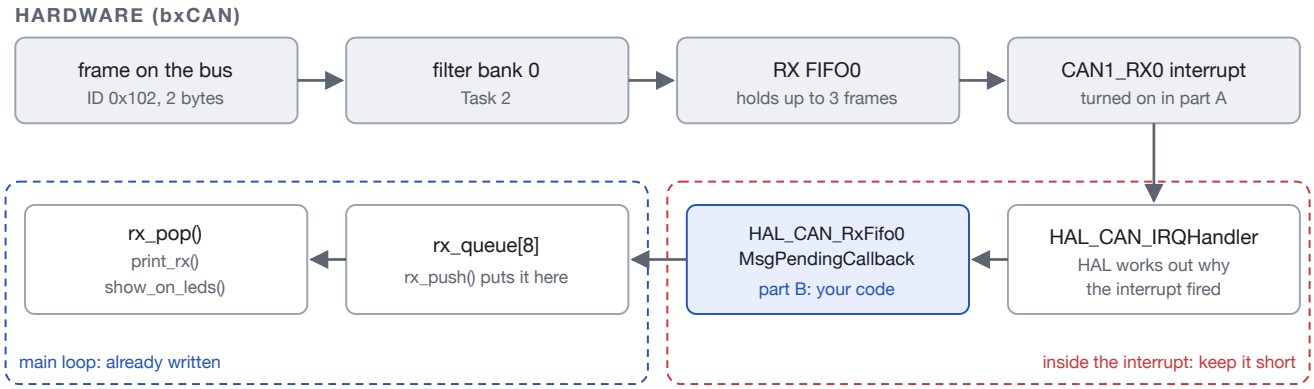
- Every send prints “failed”: Task 2 isn't working, so the controller isn't on the bus.
- The test complains about the length: `DLC` must be 2, the number of data bytes.
- The data shows `00 00`: you passed an empty array, or the wrong variable.

Task 4: Receive frames

Goal: every frame another board sends is printed as an `rx` line and shown on the LEDs.

You'll use: `HAL_CAN_ActivateNotification()`, `HAL_CAN_GetRxMessage()`, `CAN_RxHeaderTypeDef`, `rx_push()`.

What you need to know



The path of a received frame. Hardware does the top row; you write the blue box. The rest is already in `main.c`.

- An **interrupt** pauses `main()` wherever it is, runs a handler, then carries on. The HAL turns the CAN1 RX0 interrupt into a call to `HAL_CAN_RxFifo0MsgPendingCallback()`. You don't call this function yourself; the HAL does, every time a frame lands in FIFO0.
- **Keep interrupt code short.** Printing a 40-character line at 115200 baud takes about 3.5 ms, and other interrupts wait meanwhile; on a robot that could mean missed motor feedback. So the callback only copies the frame into a queue with `rx_push()`, and the main loop prints it later.
- **The interrupt keeps firing while FIFO0 is not empty.** If the callback doesn't call `HAL_CAN_GetRxMessage()`, the frame stays there, the interrupt fires again straight away, and the board is stuck in it forever. That's why you write part B before part A.

Steps

1. **Part B, the callback.** In `HAL_CAN_RxFifo0MsgPendingCallback()`, make a `CAN_RxHeaderTypeDef` and a `uint8_t data[8]`, call `HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &header, data)`, and if it returns `HAL_OK`, call `rx_push(&header, data)`. Use the `hcan` parameter, not `&hcan1`; it's the same handle, but this is how HAL callbacks are written.
2. **Part A, the switch.** At the end of `can_start()`, call `HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING)`, with the usual `HAL_OK` check.

Check

```
[node1] tx id=0x101 dlc=2 data: 01 00
[node2] rx id=0x101 dlc=2 data: 01 00
[node3] rx id=0x101 dlc=2 data: 01 00
[node2] tx id=0x102 dlc=2 data: 02 00
[node1] rx id=0x102 dlc=2 data: 02 00
[node3] rx id=0x102 dlc=2 data: 02 00
```

Every frame is received by both other boards and never by its sender. That's CAN broadcasting (2.4). The LEDs now show the counter of the last frame received. `make test`: all five tests `OK`, ending with `Tests finished successfully :)`.

COMMON MISTAKES

- **node2 and node3 stop after 0.5 s** and only node1 keeps sending: you turned the interrupt on (part A) but the callback doesn't read the frame out. node2 and node3 are stuck re-entering the interrupt.
- `tx` lines but no `rx` lines: part A is missing, or the callback's name is misspelled. It must be exactly `HAL_CAN_RxFifo0MsgPendingCallback`, or the HAL calls its own empty version instead.

DONE!

You've configured a CAN controller, sent frames, and received them through an interrupt, the same steps that talk to every motor and board on the robot. Commit your work, then try the challenges.

7 Challenges

Optional, once all four tasks pass. These change what's on the bus, so some automatic tests will fail afterwards; that's expected. Commit your working Task 4 first so you can get back to it.

Challenge 1: Send a 16-bit number

Make each heartbeat 4 bytes long. Bytes 2 and 3 hold a 16-bit value, `NODE_ID × 1000 + counter`, **high byte first** ("big-endian", the order DJI motors use). Receivers print the decoded value.

- `value >> 8` is the high byte and `value & 0xFF` the low byte.
- To rebuild it: `(data[2] << 8) | data[3]`. You'll need to add a line to `print_rx()` or to the main loop.

```
[node1] tx id=0x101 dlc=4 data: 01 00 03 E8
[node2] rx id=0x101 dlc=4 data: 01 00 03 E8  -> value=1000
[node2] tx id=0x102 dlc=4 data: 02 00 07 D0
[node1] rx id=0x102 dlc=4 data: 02 00 07 D0  -> value=2000
```

Think about it: 1000 is `0x03E8`. What would the receiver print if it got the byte order backwards?

Challenge 2: Filters

Make every board accept **only ID 0x101**. Change only the filter in `can_start()`.

In 32-bit mask mode the 11-bit standard ID sits in the **top 11 bits** of the 32-bit filter register:

Register bits	31 … 21	20 … 3	2	1	0
Meaning	STID[10:0], the standard ID	EXID (extended IDs only)	IDE	RTR	0
HAL field	`FilterIdHigh` holds bits 31…16 and `FilterIdLow` holds bits 15…0, so the standard ID lands in bits 15…5 of `FilterIdHigh`.				

Expected: node1 receives **nothing** (it never hears itself, and only 0x101 is allowed), and node2 and node3 receive only 0x101.

```
[node1] tx id=0x101 dlc=2 data: 01 00
[node2] rx id=0x101 dlc=2 data: 01 00
[node3] rx id=0x101 dlc=2 data: 01 00
[node2] tx id=0x102 dlc=2 data: 02 00
[node3] tx id=0x103 dlc=2 data: 03 00
```

Think about it: which mask accepts *both* 0x102 and 0x103 but nothing else? (They differ only in bit 0.)

Challenge 3: A DJI-style motor command

DJI's C620 motor controllers listen for **ID 0x200**: 8 bytes holding four signed 16-bit current commands, for motors 1 to 4, each high byte first (Appendix A).

- node1 is the "main controller": right after each heartbeat it also sends ID 0x200 with `1000, −1000, 0, 0`. Write a `send_motor_command(int16_t m1, int16_t m2, int16_t m3, int16_t m4)` and call it only when

`NODE_ID == 1`.

- node2 pretends to be motor 1 (bytes 0–1) and node3 motor 2 (bytes 2–3). In the main loop, when a frame with ID 0x200 arrives, each decodes *its own* two bytes into an `int16_t` and prints it.

```
[node2] rx id=0x200 dlc=8 data: 03 E8 FC 18 00 00 00 00
[node3] rx id=0x200 dlc=8 data: 03 E8 FC 18 00 00 00 00
[node2] my current command = 1000
[node3] my current command = -1000
```

Think about it: −1000 arrives as `FC 18`. Why? (Look up “two's complement”.) And why do the LEDs suddenly show `11101000` after a 0x200 frame?

8 Check your work automatically

`sim/can_bus.robot` is an automatic test, written for Renode's Robot Framework integration. It starts the three boards and checks them from the outside, one test per task:

Test	Passes when…
Task 1 - LEDs Show NODE_ID At Boot	right after boot, node3 has LED_A and LED_B on and LED_C off
Task 2 - CAN1 Is Started With A Filter	CAN1 has left initialization mode (`CAN_MCR`) and filter bank 0 is active (`CAN_FA1R`)
Task 3 - Each Node Sends Its Heartbeat	frames 0x101, 0x102 and 0x103 appear on the bus, 2 bytes each, starting with the node's number
Task 4 - Every Node Hears The Other Two But Never Itself	each board prints `rx` lines for the other two IDs, and never for its own
Task 4 - LEDs Show The Received Counter In Binary	after node2 receives counter 1, LED_A is on and LED_B is off

```
make test
```

Each test ends with `OK` , or a failure with the reason. When all five pass, the run ends with `Tests finished successfully :)` . Details, including the simulator log of each test, are written to `log.html` ; open it in a browser.

EXPECTED FAILURES

On the starter every test fails. After Task 1, only Task 1 passes, and so on. After a challenge, some tests may fail again because the bus traffic changed; that's fine.

ON A MAC

`renode-test` needs a few Python packages the first time: `python3 -m pip install -r /Applications/Renode.app/Contents/MacOS/tests/requirements.txt` . Codespaces and `setup.sh` already handle this.

If you push your work to your own copy of the repository on GitHub, the same tests run there automatically (`.github/workflows/ci.yml`); a green check mark means every task passes.

9 From the simulator to real boards

9.1 What the simulator does not check

Renode models CAN at the level of whole frames. Several things that break real buses are invisible in the simulator:

On real hardware	In Renode
CAN1 must be on **PD0/PD1** (the pins wired to the transceiver)	Pin choice is ignored. PA11/PA12 would “work” too.
`CAN1_RX` needs a pull-up	Not modeled
All boards must use the **same bit rate**	Bit timing is stored but ignored. A board at 500 kbit/s still “works”.
Someone must **ACK**; errors, retries and bus-off happen	Every frame succeeds instantly, even with no other board.
Arbitration takes time; a busy bus delays frames	Frames are delivered instantly, no contention.
Wiring, termination, loose connectors, noise	Not modeled

The repository also works around one Renode quirk: ST's `HAL_CAN_Init()` requests initialization mode while the controller is still asleep, which real bxCAN allows and Renode's model didn't. A one-line hook in `sim/dev-a.repl` fixes that. You don't need it on hardware.

9.2 What you need

- 2 or 3 Dev-A boards, powered the way your team normally does it
- CAN cables between the boards' **CAN1** ports (CAN_H to CAN_H, CAN_L to CAN_L)
- An ST-Link (SWD) to flash, and a 3.3 V USB-to-serial adapter to read USART6

9.3 Wiring checks

- **Termination.** With everything powered off, measure the resistance between CAN_H and CAN_L. About **60 Ω** (two 120 Ω resistors in parallel) is right. The Dev-A schematic shows a 120 Ω resistor next to each MCP2562 transceiver, so check how many are actually connected on your bus.
- **Serial.** USART6 is TX = PG14, RX = PG9 on the MCU. Find which connector they go to in the Dev-A schematic. Connect adapter RX to board TX, adapter TX to board RX, and GND to GND. Open the port at **115200 baud, 8N1**.

9.4 Flash each board

Each board gets its own file. With STM32CubeProgrammer's command-line tool:

```
STM32_Programmer_CLI -c port=SWD -w build/node1.elf -v -rst     # first board
STM32_Programmer_CLI -c port=SWD -w build/node2.elf -v -rst     # second board
```

You should see the same `tx` / `rx` lines as in the simulator, and the LEDs counting in binary.

9.5 Rebuild this project in CubeMX

Your team's projects start in STM32CubeMX. To recreate this one (good practice!), create a new project for **STM32F427IIHx** and set:

Where in CubeMX	Setting
System Core → RCC	High Speed Clock (HSE): Crystal/Ceramic Resonator
Clock Configuration	Input 12 MHz, PLL source HSE, /M = 6, ×N = 180, /P = 2, SYSCLK = PLLCLK (180 MHz), APB1 ÷4 (45 MHz), APB2 ÷2 (90 MHz)
Connectivity → CAN1	Activated. Move the pins to **PD0** (CAN1_RX) and **PD1** (CAN1_TX). GPIO Settings: PD0 pull-up. Parameters: Prescaler 3, Time Quanta in Bit Segment 1 = 11, Bit Segment 2 = 3, ReSynchronization Jump Width = 1; check that Baud Rate shows 1000000. NVIC Settings: enable “CAN1 RX0 interrupt”.
Connectivity → USART6	Asynchronous, 115200 baud. Pins **PG14** (TX) and **PG9** (RX).
Pinout view	PG1–PG8: GPIO_Output, labels LED_A…LED_H. GPIO settings: output level **High** (LEDs off).
Project Manager	Toolchain: CMake or Makefile. Generate code.

Then copy everything between the `USER CODE` markers from your finished `main.c` into the generated `main.c`.

9.6 First-time hardware problems

Symptom	Likely cause
`tx skipped: all 3 mailboxes busy`	Nobody ACKs: the other board isn't running, isn't wired to CAN1, uses a different bit rate, or wrong pins in CubeMX.
Nothing at all on the serial port	TX/RX swapped, missing GND, wrong baud rate, or stuck in `Error_Handler()` (check with the debugger).
Works for a while, then stops	Bus-off from errors: check termination and connectors.

10 Troubleshooting

Message	Fix
`arm-none-eabi-gcc: No such file or directory`	The compiler isn't on your `PATH`. Pass it: `make PREFIX=/path/to/bin/arm-none-eabi-`.
`bad CPU type in executable` (Mac)	You're using an Intel-only compiler (e.g. from STM32CubeIDE). Use the arm64 ARM GNU toolchain (chapter 3).
`renode: command not found`	`make sim RENODE=/path/to/renode`
Renode can't download `STM32F40x.svd.gz`	The first run needs internet. Try again on a working connection.
Only the three `up:` lines, then nothing	Expected before Task 3. After Task 3, check that Task 2 works and that `send_heartbeat()` really calls `HAL_CAN_AddTxMessage()`.
No `up:` lines at all	The board stopped in `Error_Handler()` before printing: a HAL call in `can_start()` didn't return `HAL_OK`. To see where a board is stuck, in the Renode prompt: `pause`, `mach set "node1"`, `cpu PC`, then look up the address with `arm-none-eabi-addr2line -f -e build/node1.elf 0x…`
Every send prints “failed”	CAN1 isn't on the bus: Task 2 isn't done, or `HAL_CAN_Start()` isn't reached.
`tx` lines but no `rx` lines	Task 4 part A is missing, the callback's name is misspelled, or your filter rejects everything.
node2 and node3 go silent after 0.5 s; only node1 keeps sending	The RX interrupt is on but the callback doesn't call `HAL_CAN_GetRxMessage()`, so the interrupt fires forever (Task 4).
`renode-test: command not found`	Renode's folder isn't on your `PATH` (chapter 3), or pass it: `make test RENODE_TEST=/path/to/renode-test`.
Compile error after a change	Read the *first* error line; later ones are often caused by it. Missing prototype? Missing semicolon?

A DJI motor CAN frames

This is what challenge 3 was preparing you for. All values are signed 16-bit, **high byte first**, at 1 Mbit/s. Always check the official manual for your motor and ESC (e.g. the C620 and GM6020 user guides); this table is a summary.

C620 / C610 ESCs (M3508, M2006 motors)

Direction	ID	Data
Command (you send)	`0x200`	bytes 0–1: motor 1 · 2–3: motor 2 · 4–5: motor 3 · 6–7: motor 4. C620 current command −16384…16384 (about −20…20 A).
Command (you send)	`0x1FF`	the same layout for motors 5–8
Feedback (motor sends)	`0x200 + motor ID` (0x201–0x208)	bytes 0–1: rotor angle 0…8191 (= 0…360°) · 2–3: speed in rpm · 4–5: actual torque current · 6: temperature °C (C620) · 7: unused

GM6020 gimbal motor

Direction	ID	Data
Command	`0x1FF` / `0x2FF`	four voltage commands (−30000…30000) for motor IDs 1–4 / 5–7
Feedback	`0x204 + motor ID`	same layout as the C620 feedback

WATCH OUT

Command ID `0x1FF` is shared: C620 motors 5–8 and GM6020 motors 1–4 both listen to it. Two devices must never decode the same bytes on the same bus, so plan your motor IDs.

Reading feedback is exactly the decoding you did in challenges 1 and 3: check the ID, then combine two bytes into an `int16_t`.

B Renode cheat sheet

Type these at the Renode prompt while `make sim` is running.

Command	What it does
`pause` / `start`	Freeze / resume all three boards
`quit`	Exit Renode
`mach set "node2"`	Choose which board the next commands talk to
`gpioPortG.led_a State`	Is LED_A on (`True`) or off (`False`)?
`cpu PC`	Where is this board's CPU right now?
`sysbus ReadDoubleWord 0x4000641C`	Read a register directly (this one is `CAN1_BTR`)
`emulation RunFor "00:00:02"`	Run for exactly 2 s of simulated time, then pause
`machine Reset`	Reset the selected board (like pressing its reset button)
`help`	List everything

How the simulation is put together

`sim/dev-a.repl` describes one board: it starts from Renode's generic STM32F4 file and adds the parts this project uses (USART6, the 8 LEDs, the 180 MHz SysTick, the CAN init fix). `sim/can_bus.resc` builds the bus:

```
emulation CreateCANHub "canHub"

mach create "node1"
machine LoadPlatformDescription $ORIGIN/dev-a.repl
sysbus LoadELF $ORIGIN/../build/node1.elf
connector Connect sysbus.can1 canHub
showAnalyzer sysbus.usart6 Antmicro.Renode.Analyzers.LoggingUartAnalyzer
… (the same for node2 and node3)
```

Want a fourth board? Copy one block, call it `node4` , add `4` to `NODES` in the Makefile, and run `make sim` .

Further reading

- STM32F427 reference manual (RM0090), chapter “Controller area network (bxCAN)”
- ST's HAL manual UM1725, section “HAL CAN Generic Driver”
- Renode documentation: renode.readthedocs.io
- Dev-A schematic and user manual from your team's resources